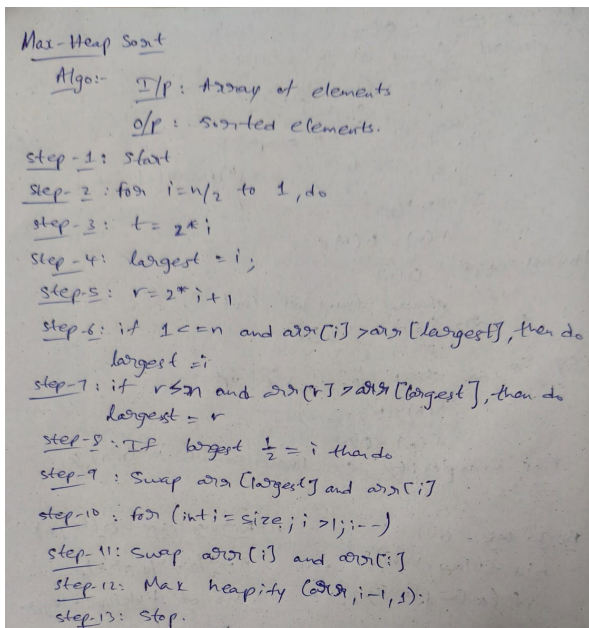


PRACTICAL-03

AIM: Implementation of max-heap sort algorithm.

1) MAX HEAP

ALGORITHM:



Max-Heap Sort
Algo:- I/p: Array of elements
O/p: Sorted elements.
Step-1: Start
Step-2: for $i = n/2$ to 1, do
Step-3: $l = 2 * i$
Step-4: $largest = i$
Step-5: $r = 2 * i + 1$
Step-6: if $l \leq n$ and $arr[l] > arr[largest]$, then do
 $largest = l$
Step-7: if $r \leq n$ and $arr[r] > arr[largest]$, then do
 $largest = r$
Step-8: If $largest \neq i$ then do
Step-9: Swap $arr[largest]$ and $arr[i]$
Step-10: for ($int i = size; i > 1; i--$)
Step-11: Swap $arr[i]$ and $arr[1]$
Step-12: Max heapify ($arr, i-1, 1$)
Step-13: Stop.

PROGRAM:

```
#include <iostream>

using namespace std;

void maxheapify(int arr[], int size, int i)
{
    int largest = i;

    int l = 2 * i;

    int r = 2 * i + 1;

    if (l <= size && arr[l] > arr[largest])
    {
        largest = l;
    }

    if (r <= size && arr[r] > arr[largest])
```

```
{
    largest = r;
}
if (largest != i)
{
    swap(arr[largest], arr[i]);
    maxheapify(arr, size, largest);
}
}

void heapsort(int arr[], int size)
{
    for (int i = size / 2; i >= 1; i--)
    {
        maxheapify(arr, size, i);
    }
    for (int i = size; i > 1; i--)
    {
        swap(arr[1], arr[i]);
        maxheapify(arr, i - 1, 1);
    }
}

int main()
{
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n + 1];
    cout << "Enter elements: ";
    for (int i = 1; i <= n; i++)
    {
        cin >> arr[i];
    }
}
```

```
heapsort(arr, n);

cout << "Sorted array: ";

for (int i = 1; i <= n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}
```

OUTPUT:

```
Output
Enter number of elements: 5
Enter elements: 5
9
7
6
1
Sorted array: 1 5 6 7 9
=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time Complexity:

for $i = n/2, i \geq 1, i-- \rightarrow n$
 $\text{max_heapify(arr, n, i)} \rightarrow \log n$

for $i = n, i > 1, i-- \rightarrow n$
 $\text{swap(arr[i], arr[i-1])} \rightarrow 1$
 $\text{Max_heapify(arr, i-1, i)} \rightarrow \log n$

$T(n) = n \times \log n + n + 1 \times \log n$
 $= n \log n + n \log n$
 $T(n) = O(n \log n)$

Best case : $T(n) = O(n \log n)$
Average case : $T(n) = O(n \log n)$
Worst case : $T(n) = O(n \log n)$

Best Case :
for $i = n/2$ to 1
 $\text{max_heapify(arr, n, i)} = n$
for $i = n$ to 2
 $\text{swap(arr[i], arr[i-1])}$
 $\text{max_heapify(arr, i-1, i)} \rightarrow \log n$
 $T(n) = O(n) + n \times O(\log n)$
 $T(n) = O(n) + O(n \log n)$
 $T(n) = O(n \log n)$

Average Case :
for $i = n$ to 2
 $\text{swap}() \rightarrow 1$
 $\text{Max_heapify}() \rightarrow \log n$
 $T(n) = n + 1 + \log n$
 $T(n) = O(n) + O(n \log n)$
 $T(n) = O(n \log n)$

Worst Case :
Max heapify from root to lowest level of the heap
 $\Rightarrow \log_2 n$
 $\text{Max_heapify}() \rightarrow \log n$
 $T(n) = n + n \times \log n$
 $T(n) = O(n \log n)$

Conclusion: Thus, the Max Heap Sort algorithm was successfully implemented. It efficiently sorts elements using a max heap with a time complexity of $O(n \log n)$.